



算法和数据结构

第九课1-2 数论

星空创客

2022





写模板

- 素数判断的函数



重点知识

- 素数判断、最大公约数、约数和
- 埃氏筛法
- 分解质因子



一、素数相关

- 素数（质数）：其因子只有1和它本身的整数
- 常用的一个概念
- 常见的题型：
 - 判断素数
 - 素数和
 - 找出某一范围内的素数

素数的判别

- 1) 最朴素的判别方法：从2开始，依次判别比自己小的数是否能被自己整除

```
bool is_prime(int n)
{
    for (i=2; i<n; i++)
    {
        if (n%i==0) return 0;
    }
    return 1;    //1=素数
}
```

复杂度： $O(n)$

- 2) 稍聪明一些的方法：只需判断到 $n/2$

```
bool is_prime(int n)
{
    for (i=2; i<=n/2; i++)
    {
        if (n%i==0) return 0;
    }
    return 1;    //1=素数
}
```

复杂度： $n/2$ 与 n 其实没什么区别，复杂度依然是 $O(n)$

- 3) 常规的做法：只需判断到 $\sqrt{n}$

```
bool is_prime(int n)
{
    int i, k=sqrt(n);
    if(n<2) return 0;
    for(i=2; i<=k; i++)          //或者 i*i<=n;
    {
        if(n%i==0) return 0;
    }
    return 1;    //1=素数
}
```

- 复杂度： $O(\sqrt{n})$

- 
- 4) 埃式筛法：使用场景，在一个范围内找出素数
 - 利用第3种方法，如果在 n 个数里找出所有素数，需对所有数进行素数判断，复杂度 $O(n\sqrt{n})$
 - 埃氏筛法的原理：枚举 $2 \sim n$ 之间每个数的所有倍数，标记为合数，那么剩下的就是质数了
 - 这个复杂度能够达到 $O(n\log n)$
 - 开一个数组，代表每个数是素数(true)或合数(false)，初始化全部为true，然后进行一个 $2 \sim n$ 的循环，把合数标记为false

代码--埃氏筛法

```
#define MAXN 100000
bool prime[MAXN];    //全局数组
void shai(int n)
{
    int i;
    for(i=2;i<=n;i++) prime[i]=true;    //先全部标记1
    for(i=2;i<=n;i++)
    {
        if(prime[i])    //如果为true则处理， 如果为false则表明已经是
            合数了， 不处理
        {
            for(int j=i+i;j<=n;j+=i) //将其倍数标记， j从i的2倍开始，
            一直到n全部做标记
            {
                prime[j]=false;    //标记此数为合数
            }
        }
    }
}
```

我们找2-20这个范围，用调试看看数组的变化，理解其工作原理



2040: 【例5.7】筛选法找质数



课堂练习 P3912 素数个数

- 体会埃氏筛法
- 体会优化后的效果



```
#include <bits/stdc++.h>
using namespace std;
bool prime[100000005];
int sum=0;
void shai(int n)
{
    int i;
    for(i=2;i<=n;i++) prime[i]=true;
    for(i=2;i*i<=n;i++)          //优化后100分
    {
        if(prime[i])
        {
            for(int j=i*i;j<=n;j+=i)//筛掉合数， 优化
            {
                prime[j]=0;
            }
        }
    }
    for(i=2;i<=n;i++) if(prime[i]) sum++;          //重新找一遍素数， 优化
}
```



```
int main()
{
    int n,m,i,x;
    cin>>n;
    shai(n);
    cout<<sum;
    return 0;

}
```



分解质因子

- 每一个合数都可以几个质数相乘的形式，其中每个质数都是这个合数的因子，称为“质因数”或“质因子”。把一个合数用质数相乘的形式表示出来，称为分解质因数。
- 例如： $12=2 \times 2 \times 3$

2032: 【例4.18】分解质因数

【题目描述】

把一个合数分解成若干个质因数乘积的形式(即求质因数的过程)叫做分解质因数。分解质因数(也称分解素因数)只针对合数。

输入一个正整数 n ，将 n 分解成质因数乘积的形式。

【输入】

一个正整数 n 。

【输出】

分解成质因数乘积的形式。质因数必须由小到大，见样例。

【输入样例】

36

【输出样例】

$36=2*2*3*3$

思路

- 可以发现，一个分解式中，一个质因子可能出现不止一次，例如 $60=2*2*3*5$ 这个式子，2出现了2次，这告诉我们如果找到一个因子，还不能马上去找更大的因子，而是要反复确认在除以因子之后的商中，是否还有这个因子，这样的处理，刚好保证了分解式中的所有因子是质因子，比如当2进行了2次分解后，因子4便不会出现在分解式中。



```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    int n,i,flag=0;
    cin>>n;
    cout<<n<<"=";
    for(i=2;i<=n;i++) //从2开始枚举质因子
    {
        while(n%i==0) //反复除以这个因子直到没有
        {
            if(flag==0)
            {
                cout<<i;
                flag=1;
            }
            else cout<<"*"<<i;
            n/=i;
        }
    }
    return 0;
}
```



课堂练习1098：质因数分解



2137 - 质因子2



```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    int n,i;
    cin>>n;
    for(i=2;i<=n/i;i++) //降低循环次数，根号n复杂度
    {
        while(n%i==0)
        {
            cout<<i<<endl;
            n/=i;
        }
    }
    if(n>1) cout<<n<<endl;    //最后再判断一次
    return 0;
}
```



28单元 习题一

- 二进制数 11.01 在十进制下是（ ）。
- A. 3.01
- B. 3.05
- C. 3.125
- D. 3.25

习题二

- 一个二维数组定义为 `double array[3][10];`，则这个二维数组占用内存的大小为（ ）字节。
- A. 30
- B. 60
- C. 120
- D. 240

习题三

- 下列关于进制的叙述，不正确的是（ ）。
- A. 正整数的二进制表示中只会出现 0 和 1。
- B. 10 不是 2 的整数次幂，所以十进制数无法转换为二进制数。
- C. 从二进制转换为 8 进制时，可以很方便地由低到高将每 3 位二进制位转换为对应的一位 8 进制位。
- D. 从二进制转换为 16 进制时，可以很方便地由低到高将每 4 位二进制位转换为对应的一位 16 进制位。

习题四

- 已知 `char a; float b; double c;`
- 执行语句 `c=a+b+c;` 后变量 `c` 的类型是（ ）。
- A. `char`
- B. `float`
- C. `double`
- D. `int`

习题五

- 填空:
- `#include <iostream>`
- `using namespace std;`
- `int main() {`
- `int i = 100, x = 0, y = 0;`
- `while (i > 0) {`
- `i--;`
- `x = i % 8;`
- `if (x == 1)`
- `y++;`
- `}`
- `cout << y << endl;`
- `return 0;`
- `}`
- 输出: _____



答案

- 1) D
- 2) D
- 3) B
- 4) C
- 5) 13

1-99间有多少个数模8的余数为1

上节回顾 筛法求素数

- 自主练习
- 2040: 【例5.7】筛选法找质数
- 东方:
- 1838 - 【基础】分解质因数
- **2136 - 筛素数**
- 1234 - 任意输入一正整数N, 要求把它拆成质因子的乘积



约数相关

- 约数（因数，因子）：能被 n 整除的数，成为 n 的约数，1和自己本身也是自己的约数
- 常用场景：最大公约数，约数和，完全数，亲和数，质因数分解等
- 相关内容我们在春季班已经学过，请大家翻阅相关课件和练习进行复习

最大公约数的求法

- 1) 穷举法：按照概念模拟就行

```
int gcd(int m, int n)
{
    if (m < n) {t=n; n=m; m=t};
    for (i=n; i>0; i--)
    {
        if (m%i==0 && n%i==0) return i;    //
    }
}
```

这种方法显然很笨，时间复杂度 $O(n)$

- 2) 更相减损法：更相减损术， 出自于中国古代的《九章算术》，也是一种求最大公约数的算法。
- ①先判断两个数的大小，如果两数相等，则这个数本身就是它的最大公约数。
- ②如果不相等，则用大数减去小数，然后用这个较小数与它们相减的结果相比较，如果相等，则这个差就是它们的最大公约数，而如果不相等，则继续执行②操作。

```
int gcd(int m, int n)
{
    while (m != n)
    {
        if (m > n) m -= n;           //如果m>n, 则m替换为两者之差
        else      n -= m;           //如果n>m, 则n替换为两者之差
    }
    return m; //如果相等, 则m (或n) 就是最大公约数
}
```

- 3) 辗转相除法（欧几里得法）：用大数除以小数得到余数，如果余数为0，则最大公约数就是小数；否则将余数换成小数，继续循环，直到余数为0

//最大公约数（非递归算法）

```
int gcd( int m , int n)
{
    int t;
    while( m % n )
    {
        t = m % n;
        m = n;
        n = t;
    }
    return n;
}
```



//最大公约数（递归算法） 重要！ ！ ！

```
int gcd( int m , int n)
{
    if( n == 0 )
        return m;
    else
        return gcd(n,m%n);
}
```




1088 - 【入门】求两个自然数M和N的最大公约数



自主练习

- 1207: 求最大公约数问题
- 1087 – 【入门】两个自然数M和N的最小公倍数。
- 1099: 第 n 小的质数

课堂练习 P1888 三角函数

题目描述

输入一组勾股数 a, b, c ($a \neq b \neq c$)，用分数格式输出其较小锐角的正弦值。（要求约分。）

输入格式

一行，包含三个正整数，即勾股数 a, b, c （无大小顺序）。

输出格式

一行，包含一个分数，即较小锐角的正弦值

输入 #1

3 5 4

输出 #1

3/5



思路

- 所谓正弦，就是直角三角形短边除以斜边
- 已知 a 、 b 、 c 是整数且是勾股值，所以找出最大最小后，相除结果，除以GCD即可达到约分
- 只有3个数，找最大最小可用多种方法



```
#include <bits/stdc++.h>
using namespace std;
int gcd(int m,int n)
{
    if(n==0) return m;
    return gcd(n,m%n);
}
int main()
{
    int a,b,c,maxx,minn;
    cin>>a>>b>>c;
    maxx=max(a,max(b,c));    //最大值
    minn=min(a,min(b,c));    //最小值
    cout<<minn/gcd(maxx,minn)<<"/"<<maxx/gcd(maxx,minn);
    return 0;
}
```

1915: 【01NOIP普及组】最大公约数与最小公倍数

题目描述

输入两个正整数 x_0, y_0 ，求出满足下列条件的 P, Q 的个数：

1. P, Q 是正整数。
2. 要求 P, Q 以 x_0 为最大公约数，以 y_0 为最小公倍数。

求：满足条件的所有可能的 P, Q 的个数。

输入格式

一行两个正整数 x_0, y_0 。

输出格式

一行一个数，表示求出满足条件的 P, Q 的个数。

输入

3 60

输出

4



```
#include <iostream>
using namespace std;
int gcd(int m,int n)
{
    if(n==0) return m;
    else return gcd(n,m%n);
}
int lcm(int m,int n)
{
    return m/gcd(m,n)*n;
}
int main()
{
    int x,y,sum=0;
    int i,j;
    cin>>x>>y;
    for(i=x;i<=y;i+=x)
    {
        for(j=y;j>=x;j-=x)
        {
            if(gcd(i,j)==x && lcm(i,j)==y) sum++;
        }
    }
    cout<<sum<<endl;
    return 0;
}
```

优化的代码单层循环

```
#include <iostream>
using namespace std;
int gcd(int m,int n)
{
    if(n==0) return m;
    return gcd(n,m%n);
}
int lcm(int m,int n)
{
    return m/gcd(m,n)*n;
}
int main()
{
    int x,y,p,q,sum=0;
    cin>>x>>y;
    for(p=x;p<=y;p+=x)
    {
        q=x*y/p;
        if(gcd(p,q)==x && lcm(p,q)==y)
        {
            sum++;
        }
    }
    cout<<sum;
    return 0;
}
```




上海月赛202312 丙3 数轴旅行



```
//若d能整除ai的gcd则yes
#include <bits/stdc++.h>
using namespace std;
long long gcd(long long m,long long n)
{
    if(n==0) return m;
    else return gcd(n,m%n);
}
int main()
{
    long long n,i,d,sum=0,k,a;
    cin>>n>>d;
    cin>>k;
    for(i=2;i<=n;i++)
    {
        scanf("%lld",&a);
        k=gcd(k,a);
    }
    if(d%k==0) cout<<"Yes";
    else cout<<"No";
    return 0;
}
```

1899: 【17NOIP提高组】小凯的疑惑

【题目描述】

小凯手中有两种面值的金币，两种面值均为正整数且彼此互素。每种金币小凯都有无数个。在不找零的情况下，仅凭这两种金币，有些物品他是无法准确支付的。现在小凯想知道在无法准确支付的物品中，最贵的价值是多少金币？注意：输入数据保证存在小凯无法准确支付的商品。

【输入】

输入数据仅一行，包含两个正整数 a 和 b ，它们之间用一个空格隔开，表示小凯手中金币的面值。

【输出】

输出文件仅一行，一个正整数 N ，表示不找零的情况下，小凯用手中的金币不能准确支付的最贵的物品的价值。

【输入样例】

3 7

【输出样例】

11

思路

- 两个互质的数无法凑成的最大数
- 无法满足 $x=ma+nb$ 的最大 x
- 例如3和7互质，无法凑成的有1、2、4、5、8、11，最大为11
- 公式： $a*b-a-b$
- 不能凑成数的个数为 $(a-1)(b-1)/2$



```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    long long a,b;
    cin>>a>>b;
    cout<<a*b-a-b;
    return 0;
}
```



约数和

- 给一个自然数 n ，求 n 的所有约数之和（不包括 n 本身）
- 例如：6的约数 $1+2+3$
- 扩展的用途：完全数，亲和数
- 下面建一个函数，来返回 n 的约数和

函数代码

```
int sum_yueshu(int n)
{
    int i, sum=0;
    for (i=1; i<=n/2; i++)
    {
        if (n%i==0)
        {
            sum +=i;
        }
    }
    return sum;
}
```

此代码时间复杂度 $O(n)$,
有没有更快的方法?

如果 i 是 n 的约数, 那么
 n/i 必然也是约数, 就可
以同时加进去, 这样查
找范围就可以控制在 $\sqrt{n}$
(n) 内, 同时要考虑如
果 $i=\sqrt{n}$, 多加了一次
 n/i 要把它减去
复杂度变为 $O(\sqrt{n})$

//O($\sqrt{n}$)的做法

```
int sum_yueshu(int n)
{
    int i,sum=1;
    for(i=2;i*i<=n;i++)           //相当于sqrt (n)
    {
        if(n%i==0)
        {
            sum+=i;
            sum+=n/i; //把n/i也加上
        }
        if(i*i==n)
        {
            sum -=i;    //如果刚好是平方根，要减去1个
        }
    }
    return sum;
}
```




课堂练习

- 1859 – 【入门】完数判断
- 1154: 亲和数
- 1150: 求正整数2和n之间的完全数



作业

- 洛谷：
- P5436 【XR-2】缘分
- 一本通：
- 1410：最大质因子序列
- 东方：
- 1872 - 【基础】N个数的最大公约数
- 1150 - 【基础】求完全数的个数



本课要达成的目标

- 熟练掌握素数的判定，尤其是埃氏筛法的原理和代码
- 熟练掌握最大公约数的求法，尤其是欧几里得法递归
- 熟练掌握约数的判定和约数和的求法